UNIVERSIDADE FEDERAL DE SERGIPE

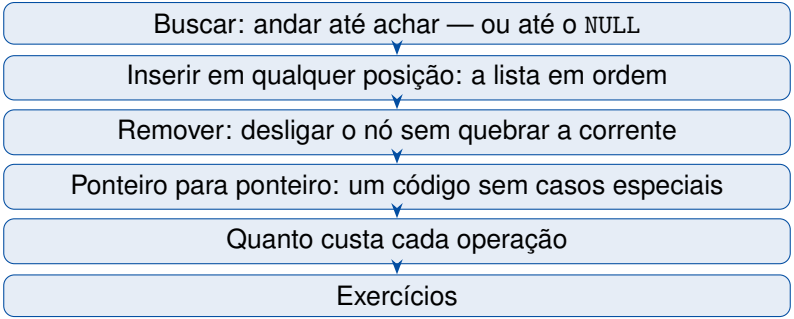
Lista encadeada: busca, inserção e remoção

Achar, pôr e tirar um nó em qualquer ponto da lista

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

O que você vai aprender hoje



A pergunta de hoje

O problema: uma lista que não é só uma pilha de valores

Até aqui

A `lista.c` da aula passada só sabia inserir *no início*, percorrer e liberar. A ordem saía invertida, e não havia como perguntar "o 20 está aí?" nem tirar um valor do meio.

O que vamos descobrir

Para achar, pôr e tirar em qualquer ponto, basta andar pelas setas — mas cada operação esconde um **caso especial** que derruba o programa se for esquecido.

A pergunta de hoje

Como buscar, inserir e remover em qualquer posição de uma lista encadeada — e quais casos especiais aparecem no caminho?

O programa de hoje: `lista.c`, versão 2 (1/3)

```c
/* lista.c --- lista em ordem crescente:
   busca, insercao e remocao */
#include <stdio.h>
#include <stdlib.h>

typedef struct no {
    int valor;
    struct no *prox;
} No;
```

```c
/* devolve o primeiro no com valor x,
   ou NULL */
No *busca(No *ini, int x)
{
    No *p = ini;
    while (p != NULL && p->valor != x)
        p = p->prox;
    return p;
}
```

`imprime` e `libera` continuam iguais aos da aula passada.

lista.c (2/3): inserir em ordem e remover

```
No *insere_ordenado(No *ini, int x)
{
    No *novo = malloc(sizeof *novo);
    if (novo == NULL) { perror("malloc"); exit
      (1); }
    novo->valor = x;
    if (ini == NULL || x < ini->valor) {
        novo->prox = ini;  /* 1o */
        return novo;
    }
    No *ant = ini;
    while (ant->prox != NULL &&
           ant->prox->valor < x)
        ant = ant->prox;
    novo->prox = ant->prox;
    ant->prox = novo;
    return ini;
}
```

```
No *remove_valor(No *ini, int x)
{
    No *ant = NULL, *p = ini;
    while (p != NULL && p->valor != x) {
        ant = p;
        p = p->prox;
    }
    if (p == NULL)     /* nao achou */
        return ini;
    if (ant == NULL)   /* 1o */
        ini = p->prox;
    else
        ant->prox = p->prox;
    free(p);
    return ini;
}
```

lista.c (3/3): o main

```c
int main(void)
{
    int entrada[] = {30, 10, 40, 20};
    No *lista = NULL;
    for (int i = 0; i < 4; i++)
        lista = insere_ordenado(lista, entrada[i]);
    imprime(lista);
    printf("busca 20: %s\n", busca(lista, 20) ? "achou" : "
     nao achou");
    printf("busca 25: %s\n", busca(lista, 25) ? "achou" : "
     nao achou");
    lista = remove_valor(lista, 10);
    lista = remove_valor(lista, 40);
    lista = remove_valor(lista, 99);
    imprime(lista);
    libera(lista);
    return 0;
}
```

Os valores chegam fora de ordem: 30, 10, 40, 20.

Depois de duas buscas, o programa tenta remover três valores — um deles não está na lista.

O que este programa imprime?

Antes de compilar, escreva sua resposta

1. Em que ordem os valores aparecem na primeira linha? Compare com a aula passada.
2. O que as duas buscas respondem?
3. Qual é a última linha impressa?
4. Dos três `remove_valor`, qual passa pelo `if (ant == NULL)`? E qual não chega a chamar `free`?

Compile, execute e compare

```
$ gcc -Wall -Wextra lista.c -o lista
$ ./lista
10 -> 20 -> 30 -> 40 -> NULL
busca 20: achou
busca 25: nao achou
20 -> 30 -> NULL
```

O que mudou

A lista saiu **em ordem**, embora os valores tenham chegado embaralhados: cada um foi encaixado no lugar certo.

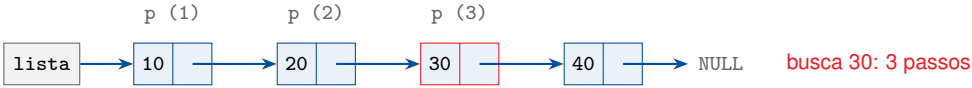
Remover o 10 mudou o início; remover o 99 não fez nada. Cada função trata esses casos com um `if` — e é deles que esta aula trata.

Buscar

Definição: busca sequencial

Busca sequencial numa lista

Começando no início, compare cada nó com o valor procurado e siga o `prox`. Pare ao **achar** (devolva o nó) ou ao chegar ao `NULL` (devolva `NULL`: não está na lista).



Intuição

Não há índice: a lista só se deixa ler na ordem das setas. Para saber que 25 *não* está lá, é preciso olhar todos os nós.

Fonte: Cormen et al. (2009), §10.2, LIST-SEARCH [2]; Celes et al. (2004), cap. 10 [1]

E se a condição vier na outra ordem?

```c
/* busca_errada.c */
No *busca(No *ini, int x)
{
    No *p = ini;
    while (p->valor != x && p != NULL)
        p = p->prox;
    return p;
}
```

A mesma condição, só que com os dois testes trocados de lugar. O `&&` não é comutativo?

Pense antes de passar o slide

1. `busca(lista, 20)` ainda funciona?
2. E `busca(lista, 25)`? Siga `p` até o fim.
3. Faz diferença compilar com `-O2`?

Execute e compare

```
$ gcc -Wall -Wextra busca_errada.c -o busca_errada      # nenhum aviso
$ ./busca_errada
10 -> 20 -> 30 -> 40 -> NULL
busca 20: achou
Segmentation fault: 11
$ gcc -O2 -Wall -Wextra busca_errada.c -o busca_errada-O2
$ ./busca_errada-O2
10 -> 20 -> 30 -> 40 -> NULL
busca 20: achou
busca 25: achou
20 -> 30 -> NULL
```

Sem otimização, o programa cai. Com `-O2`, ele não cai — e afirma que 25 está na lista.

A resposta: o `&&` para no primeiro falso

`a && b` avalia `a` primeiro; se `a` é falso, `b` **nem é avaliado** (curto-circuito). A ordem dos testes é parte do significado.

Na busca por 25, `p` chega a `NULL`. Na versão certa, `p != NULL` falha e `p->valor` nunca é lido. Na trocada, `p->valor` lê o endereço 0: *segmentation fault*.

E o `-O2`?

Ler `p->valor` com `p == NULL` é comportamento indefinido. O compilador pode supor que isso nunca acontece — e, se `p` nunca é `NULL`, o teste `p != NULL` e o "não achou" podem ser descartados.

Proteja sempre o acesso: `p != NULL && p->...` — nessa ordem.

Fonte: ISO/IEC 9899:2018, §6.5.13 e §3.4.3 [4]; Kernighan & Ritchie (1989), §2.6 [5]; medido com `busca_errada.c`

Recapitulando: buscar

O que aprendemos

- buscar é percorrer até achar o valor ou chegar ao `NULL`;
- a função devolve o nó encontrado — ou `NULL`;
- `p != NULL` vem *antes* de `p->valor`: o `&&` para no primeiro falso;
- com um ponteiro nulo, o programa pode cair — ou, otimizado, dar uma resposta errada sem aviso.

Inserir em qualquer posição

Onde encaixar o 25?

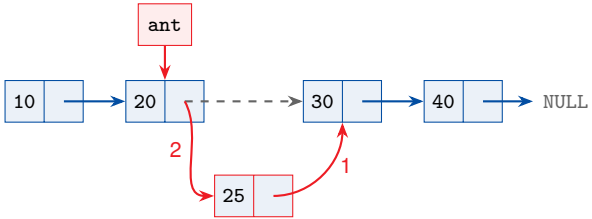
```
No *ant = ini;
while (ant->prox != NULL &&
       ant->prox->valor < x)
    ant = ant->prox;
novo->prox = ant->prox;
ant->prox = novo;
```

A lista é `10 -> 20 -> 30 -> 40` e `x` vale 25.

Pense antes de passar o slide

1. Em que nó `ant` para?
2. Por que o laço olha `ant->prox->valor`, um nó *à frente*, e não o próprio `ant`?
3. E se `x` for 50? E 5?

A resposta: parar no nó *anterior*



1. `novo->prox = ant->prox;` **2.** `ant->prox = novo;`

A lista só tem setas para a frente: ao chegar ao 30, já não dá para voltar ao 20 e mudar o `prox` dele. Por isso o laço espia um nó à frente e para no **anterior**.

A ordem 1–2 é a mesma da aula passada: ligar o novo antes de desviar a seta antiga.

Fonte: Celes et al. (2004), cap. 10 [1]; Feofiloff (2009), cap. 4 [3]

Quatro situações, dois caminhos no código

situação	exemplo	quem trata	muda o início?
lista vazia	25 em `NULL`	`if (ini == NULL || ...)`	sim
antes do primeiro	5 em `10 -> ...`	`if (... || x < ini->valor)`	sim
no meio	25	laço + passos 1 e 2	não
no fim	50	laço + passos 1 e 2	não

No fim, o laço para no último nó porque `ant->prox == NULL`; o passo 1 copia esse `NULL` para o novo nó, que vira o último *sem* caso especial.

Só o primeiro nó precisa de tratamento à parte: **não existe nó anterior** a ele — quem aponta para o primeiro é a variável `lista` do `main`.

Fonte: Celes et al. (2004), cap. 10 [1]; King (2008), §17.5 [6]

E inserir sempre no fim, quanto custa?

n	no início	no fim
10 000	0,0003 s	0,058 s
20 000	0,0002 s	0,147 s
40 000	0,0004 s	0,625 s
80 000	0,0007 s	2,479 s
160 000	0,0029 s	10,27 s

compilado com `-O2`

Para pôr no fim, `insere_fim` percorre a lista inteira a cada chamada:
$0 + 1 + \cdots + (n-1) \approx n^2/2$ passos.

Dobrou n, o tempo **quadruplicou** — o mesmo n^2 que o vetor pagava para inserir no começo.

Guardar também um ponteiro para o **último** nó elimina a caminhada (Exercício 1).

Fonte: Cormen et al. (2009), §10.2 [2]; medido com `fim_vs_inicio.c`

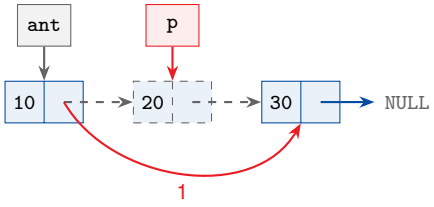
Recapitulando: inserir em qualquer posição

O que aprendemos

- para inserir, é preciso parar no nó *anterior* à posição;
- dois ponteiros, nesta ordem: `novo->prox = ant->prox; ant->prox = novo;`
- o fim não é especial; o início é, porque não tem anterior;
- inserir no fim percorrendo a lista custa n passos por inserção.

Remover

Remover: desviar a seta, depois liberar



1. `ant->prox = p->prox;`
2. `free(p);`

Para tirar o 20, o 10 tem de passar a apontar para o 30. Por isso a busca anda com **dois** ponteiros: `p`, no nó procurado, e `ant`, um passo atrás.

Na ordem contrária — `free` antes — `p->prox` seria lido de um bloco já liberado.

Fonte: Celes et al. (2004), cap. 10 [1]; Cormen et al. (2009), §10.2, LIST-DELETE [2]

Um `remove_valor` mais curto

```c
/* remove_errada.c */
No *remove_valor(No *ini, int x)
{
    No *ant = NULL, *p = ini;
    while (p != NULL && p->valor != x) {
        ant = p;
        p = p->prox;
    }
    if (p == NULL)
        return ini;
    ant->prox = p->prox;
    free(p);
    return ini;
}
```

Pense antes de passar o slide

Sumiu o `if (ant == NULL)`.

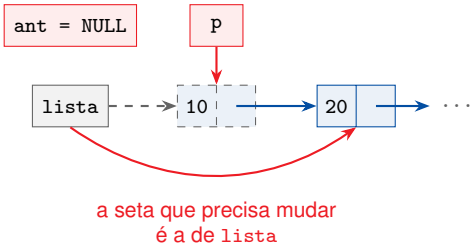
1. Remover o 40 (último) funciona?
2. E remover o 99, que não está lá?
3. E remover o 10? Quanto vale `ant` nessa hora?

Execute e compare

```
$ ./remove_errada
10 -> 20 -> 30 -> 40 -> NULL
busca 20: achou
busca 25: nao achou
Segmentation fault: 11
$ gcc -g -fsanitize=address remove_errada.c -o remove_errada-asan
$ ./remove_errada-asan
==36265==ERROR: AddressSanitizer: SEGV on unknown address 0x000000000008
==36265==The signal is caused by a WRITE memory access.
==36265==Hint: address points to the zero page.
    #0 0x0001041f8e3c in remove_valor remove_errada.c:50
    #1 0x0001041f925c in main remove_errada.c:80
```

A linha 80 é `lista = remove_valor(lista, 10);` — a primeira remoção.

A resposta: o primeiro nó não tem anterior



Se o 10 é achado de cara, o laço não roda e `ant` continua `NULL`. `ant->prox` escreve em `NULL` $+8$ — o endereço `0x8` do relatório, pois `prox` fica 8 bytes depois do início do nó.

Quem aponta para o primeiro nó não é um `prox`: é a variável `lista` do `main`. Por isso a função muda `ini` e **devolve** o novo início.

Fonte: King (2008), §17.5 [6]; Serebryany et al. (2012) [8]; medido com `remove_errada.c`

Recapitulando: remover

O que aprendemos

- remover é desviar a seta do anterior e só então liberar o nó;
- a busca anda com dois ponteiros: `ant` um passo atrás de `p`;
- três casos: não achou, achou o primeiro, achou outro;
- remover o primeiro muda o início — quem chama precisa guardar o ponteiro devolvido.

Ponteiro para ponteiro

O caso especial é sempre o mesmo

Na inserção e na remoção, o `if` extra existe pela mesma razão: a seta que precisa mudar às vezes é a variável `lista`, às vezes o `prox` de um nó.

Mas as duas coisas têm o **mesmo tipo**: `No *`. Um ponteiro do tipo `No **` pode apontar para qualquer uma delas.

Ponteiro para ponteiro

Uma variável `No **pp` guarda o endereço de um `No *`. `*pp` é esse ponteiro — dá para ler e *mudar* para onde ele aponta.

Em vez de “o nó anterior”, a busca guarda “o endereço da seta que chega ao nó atual”.

Fonte: King (2008), §17.6 [6]; Kernighan & Ritchie (1989), §5.6 [5]

ponteiro_duplo.c: sem if para o primeiro

```c
void insere_ordenado(No **pp, int x)
{
    while (*pp != NULL && (*pp)->valor < x)
        pp = &(*pp)->prox;
    No *novo = malloc(sizeof *novo);
    if (novo == NULL) {
        perror("malloc");
        exit(1);
    }
    novo->valor = x;
    novo->prox = *pp;
    *pp = novo;
}
```

```c
void remove_valor(No **pp, int x)
{
    while (*pp != NULL && (*pp)->valor != x
     )
        pp = &(*pp)->prox;
    if (*pp != NULL) {
        No *p = *pp;
        *pp = p->prox;
        free(p);
    }
}
```

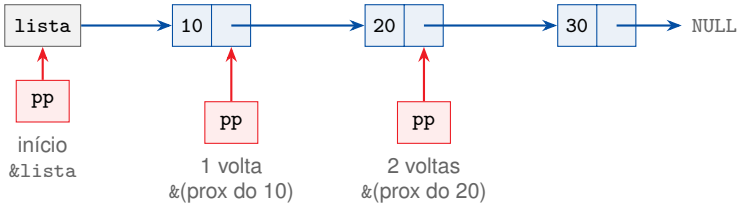
```c
/* no main */
insere_ordenado(&lista, 30);
remove_valor(&lista, 10);
```

Pense antes de passar o slide

Leia o código com calma

1. Por que o `main` passa `&lista`, e não `lista`? E por que as funções agora são `void`?
2. No começo, para onde `pp` aponta? E depois de uma volta do laço?
3. Por que `(*pp)->valor` precisa dos parênteses? E o que é `&(*pp)->prox`?
4. Remover o 10 (primeiro) e o 40 (último): siga `pp` nos dois casos.

A resposta: `pp` aponta para a seta, não para o nó



`*pp = p->prox` muda *a seta para onde `pp` aponta* — seja ela `lista` ou um `prox`. Por isso não há caso especial, e a função não precisa devolver nada: ela mexe no `lista` do `main` pelo endereço. Os parênteses são necessários porque `->` tem precedência maior que `*` e `&`: `&(*pp)->prox` é "o endereço do campo `prox` do nó atual".

Fonte: King (2008), §17.6 [6]; Kernighan & Ritchie (1989), §2.12 [5]; medido com `ponteiro_duplo.c`

Recapitulando: ponteiro para ponteiro

O que aprendemos

- o caso especial vem de a seta do primeiro nó morar fora da lista, na variável `lista`;
- `No **pp` aponta para uma seta qualquer, de dentro ou de fora da lista;
- avançar é `pp = &(*pp)->prox`; mudar a seta é `*pp = ...`;
- quem chama passa `&lista`, e a função não devolve o início.

Quanto custa cada operação

Custo das operações numa lista de n nós

operação	nós visitados (pior caso)	por quê
buscar	n	só se lê na ordem das setas
inserir no início	constante	não percorre
inserir no fim	n (constante guardando o último)	acha o último
inserir em ordem	n	acha o anterior
remover um valor	n	busca, depois desvio
remover um nó já conhecido	n	precisa achar o anterior

Encaixar e desligar um nó é barato: dois ponteiros. O caro é **chegar** ao lugar. A lista ordenada não ajuda tanto quanto o vetor ordenado: sem acesso por índice, não há busca binária.

Fonte: Cormen et al. (2009), §10.2 [2]; Knuth (1997), §2.2.3 [7]

Recapitulando: quanto custa cada operação

O que aprendemos

- ligar e desligar um nó custa o mesmo com 3 ou 3 milhões de nós;
- achar a posição custa até n passos;
- uma lista ordenada não permite busca binária;
- remover um nó conhecido ainda exige achar o anterior — o problema que a próxima aula resolve.

Mãos à obra

Altere o programa

Tarefa 1

A lista está em ordem: ao passar de `x`, a busca já pode desistir. Altere `busca` para isso. Quantos nós ela visita agora em `busca(lista, 25)`? E antes?

Tarefa 2

Insira o 20 uma segunda vez. O novo 20 entra antes ou depois do antigo? Troque < por <= no laço e preveja o que muda antes de executar.

Tarefa 3

Em `ponteiro_duplo.c`, escreva `void remove_todos(No **pp, int x)`, que tira *todas* as ocorrências de `x`. Teste com `10 -> 10 -> 20 -> 10`.

Exercícios (1/2)

Exercício 1 — inserir no fim sem percorrer

Crie `typedef struct { No *ini, *fim; } Lista;` e escreva `insere_fim(Lista *l, int x)` com custo constante. Que funções precisam atualizar `fim`? Refaça a tabela de tempos.

Exercício 2 — remover por posição

`No *remove_pos(No *ini, int k)` remove o nó da posição `k` (a partir de 0), se existir. Escreva também a versão com `No **`.

Exercício 3 — busca recursiva

`No *busca_rec(No *ini, int x)`. Quais são os dois casos base? Com 1 milhão de nós, ela funciona sem `-O2`? E com?

Fonte: Feofiloff (2009), cap. 4 [3]; Celes et al. (2004), cap. 10 [1]

Exercícios (2/2)

Exercício 4 — mover para a frente

`No *busca_mtf(No *ini, int x)`: ao achar `x`, move o nó para o início e devolve o novo início. Quais ponteiros mudam?

Exercício 5 — intercalar

`No *intercala(No *a, No *b)` junta duas listas ordenadas numa só, ordenada, **sem** `malloc`: só religando os nós existentes.

Exercício 6 — inverter no lugar

`No *inverte(No *ini)` inverte a lista mudando apenas os `prox`, sem criar nós. Quantos ponteiros auxiliares são necessários?

Fonte: King (2008), §17.5–17.6 [6]; Tenenbaum et al. (1995), cap. 4 [9]

Desafio

Uma agenda sempre em ordem alfabética

Leia comandos da entrada padrão até o fim do arquivo e mantenha uma lista de

```
typedef struct contato { char *nome; struct contato *prox; } Contato;
```

- `+ nome` insere em ordem alfabética (`strcmp`), sem repetir nomes;
- `- nome` remove; `? nome` responde “sim” ou “não”; `=` lista todos;
- cada nome mora num bloco do `malloc` com o tamanho exato — e é liberado junto com o nó.

Use `Contato **` para não ter caso especial. Com `-fsanitize=address`, nenhum erro e nenhum vazamento.

Fonte: King (2008), §17.5 e §17.6 [6]

Fechamento

O que mudou hoje

A lista deixou de crescer só pela frente: agora dá para achar um valor, encaixar um nó em qualquer ponto e tirar um nó do meio.

O que ficou

Para mexer num nó, é preciso o *anterior* — e a lista só anda para a frente.

Para levar

- buscar: `p != NULL && p->valor != x`, nessa ordem;
- inserir: `novo->prox = ant->prox; ant->prox = novo;`
- remover: `ant->prox = p->prox; free(p);`
- o primeiro nó é especial — a menos que se use `No **`.

Próxima aula: um nó que aponta também para o anterior — a lista duplamente encadeada.

Referências I


Waldemar Celes, Renato Cerqueira, and José Lucas Rangel.
Introdução a estruturas de dados: com técnicas de programação em C.
Elsevier, Rio de Janeiro, 2004.
Capítulo 10: listas encadeadas (busca, remoção e lista ordenada).


Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, Cambridge, MA, 3 edition, 2009.
Seção 10.2: linked lists (LIST-SEARCH, LIST-INSERT, LIST-DELETE).

Referências II



Paulo Feofiloff.
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.
Capítulo 4: listas encadeadas.



ISO/IEC.
Iso/iec 9899:2018 — information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seções 3.4.3 (comportamento indefinido) e 6.5.13 (operador `&&`).

Referências III

Brian W. Kernighan and Dennis M. Ritchie.
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.
Seções 2.6 (operadores lógicos), 2.12 (precedência) e 5.6 (ponteiros para ponteiros).

K. N. King.
C Programming: A Modern Approach.
W. W. Norton, New York, 2 edition, 2008.
Seções 17.5 (linked lists) e 17.6 (pointers to pointers).

Referências IV


Donald E. Knuth.
The Art of Computer Programming, Volume 1: Fundamental Algorithms.
Addison-Wesley, Reading, MA, 3 edition, 1997.
Seção 2.2.3: alocação encadeada.


Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.

Referências V



Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.
Capítulo 4: filas e listas (listas ligadas).